

KIRA AI Incident Report

Cluster Summary

The cluster is a single-node development/testing environment (`minikube`) running a microservices-based application (similar to Google's Online Boutique). The application consists of 7 deployments and 1 stateful-set-like pod (`boutique-postgres-0`).

While all deployments report `1/1` available replicas and all pods are currently in a `Running` state, a cluster-wide disturbance has occurred. Out of 8 active pods, 6 have experienced exactly **1** restart, and the `frontend` pod is exhibiting anomalous, near-zero CPU consumption.

Key Findings

- Coordinated Multi-Pod Restarts:** Six key services (`auth`, `boutique-postgres-0`, `frontend`, `order-service`, `product-service`, and `user-service`) have restarted exactly once. Only `gateway` and `orders` have 0 restarts. This point-in-time correlation points to a node-level event, resource exhaustion, or a shared dependency failure during startup.
- Anomalous Frontend CPU Metrics:** The `frontend-75bc59b865-td9rw` pod is consuming an extremely low amount of CPU (`0.000012` cores, or ~12 microcores). Compared to other idle services running at ~5-6m CPU, this suggests the frontend is either deadlocked, blocked on a network I/O call, or failing to receive any traffic/readiness checks.
- Single Point of Failure (SPOF):** Every deployment is configured with `desired\_replicas: 1`. In a single-node configuration, any resource constraint or pod crash immediately translates to service downtime.

Severity Assessment

Severity: SEV-2 (Major Incident - Degraded / Fragile State)

Justification: While the pods are currently reporting `Running`, the widespread restart event across 75% of the fleet indicates instability. The exceptionally low CPU usage on the `frontend` indicates a high probability of a "zombie" process or a broken routing layer, meaning users are likely experiencing a complete outage even though the Kubernetes control plane sees the pod as `Running`.

Root Cause Analysis

Based on the telemetry, there are two primary hypotheses for this failure pattern:

Hypothesis 1: Transient Dependency Crash & Cascading Startup Failure

During the initial deployment or a cluster restart, the microservices attempted to connect to the database (`boutique-postgres-0`) or the `auth`/`user` services before they were fully initialized and accepting traffic.

* Due to a lack of resilient connection retries or exponential backoff in the application code, these dependent pods crashed immediately on startup (`ExitCode 1`).

* Kubernetes restarted them, and on the second attempt, the database was ready, allowing them to transition to `Running`.

Hypothesis 2: Node-Level Resource Starvation (OOM Killer)

Running a multi-service stack on a single-node `minikube` instance often leads to memory constraints.

* The Linux Out-Of-Memory (OOM) killer or the Kubelet Eviction manager may have terminated multiple containers to reclaim memory.

* The `frontend` pod's near-zero CPU utilization could be a symptom of it being restarted and subsequently stuck in a deadlocked state due to unclean socket closure or unhandled exceptions during the abrupt termination.

Recommended Remediation

1. ****Investigate Last Termination Reason:**** Query the Kubelet to find out **why** the pods restarted (e.g., were they `OOMKilled` or did they exit with an error code?).
2. ****Restart the Frontend Pod:**** Since the `frontend` is in a suspected zombie state (near-zero CPU), perform a rolling restart of the frontend deployment to restore service.
3. ****Verify Database Readiness:**** Confirm that `boutique-postgres-0` is fully accepting connections and that the connection pools of dependent services are healthy.
4. ****Inspect Node Resources:**** Check if the `minikube` node is experiencing memory pressure.

Preventative Measures

1. ****Implement Readiness & Liveness Probes:**** Ensure all pods have properly configured probes with an initial delay (`initialDelaySeconds`) to allow databases and internal APIs to warm up before traffic is routed.
2. ****Establish Resilient Connection Retries:**** Refactor application code to use exponential backoff when connecting to external dependencies (like Postgres or the Auth service) instead of crashing the process immediately.
3. ****Define Resource Requests and Limits:**** Explicitly set CPU and Memory requests/limits on all deployments to prevent a single service from starving the rest of the node and triggering OOM events.
4. ****Scale Replicas for High Availability:**** In production, scale critical services (`frontend`, `auth`, `gateway`) to `replicas: 2` or more and implement Pod Anti-Affinity.

Exact kubectl Commands to diagnose and fix the issue

1. Diagnose Pod Restart Reasons

Run this command to fetch the exit code, termination reason, and OOMKilled status for the previous execution of all restarted pods:

```
```bash
kubectl get pods -o jsonpath='{range .items[*]}{.metadata.name}{`\tLast State:
"{.status.containerStatuses[*].lastState.terminated.reason}"\tExit Code:
"{.status.containerStatuses[*].lastState.terminated.exitCode}"\n"}{end}'
```
```

2. Inspect Node Events and Conditions

Check if the minikube node is under Memory, Disk, or PID pressure:

```
```bash
kubectl describe node minikube | grep -A 10 Conditions
```
```

Identify recent OOM kills or eviction events:

```
```bash
kubectl get events --sort-by='.metadata.creationTimestamp' -A | grep -iE 'OOM|Killing|Evict'
```
```

3. Check Logs for the Prior Run (Post-Mortem)

Look at the logs of the previous container instances before they restarted to pinpoint application crash reasons:

```
```bash
kubectl logs deployment/frontend -p
kubectl logs deployment/auth -p
kubectl logs statefulset/boutique-postgres-0 -p
```
```

4. Restart the Suspicious Frontend Deployment

Force-recycle the `frontend` pod to clear the suspected zombie state:

```
```bash
kubectl rollout restart deployment/frontend
```
```

5. Monitor Live CPU and Memory Utilization

Check resource consumption of all pods to ensure they are within healthy thresholds:

```
```bash
kubectl top pods
```
```